UNIFAAT 2026.2

04 - Versionamento e Git

Prof. Luan Lourenço

AULA	DATA DA AULA	CONTEÚDO
1	19/AGO	Sistema Operacional - GUI e CLI
2	26/AGO	WSL 2 - Virtualização Linux
3	02/SET	Terminal e Linguagem da Máquina
4	09/SET	Versionamento e Git
5	16/SET	Git e GitHub
6	23/SET	Revisão
	30/SET	PROVA 1º BIMESTRE
7	07/OUT	Fundamentos de Python
8	14/OUT	Estrutura de Dados
9	21/OUT	Paradigma Orientado a Objetos
10	28/OUT	OOP - Os 3 Pilares
11	04/NOV	Object Calisthenics
12	11/NOV	Arquitetura de Código
13	18/NOV	Ramificação de Carreira
14	25/NOV	Revisão
	02/DEZ	PROVA 2º BIMESTRE
	09/DEZ	PROVA SUB
	16/DEZ	EXAME

WSL e Windows: o Filesystem Compartilhado

Os dois sistemas parecem isolados, mas compartilham o mesmo disco físico.

```
# Windows
Gerencia arquivos, janelas e hardware
```

```
# Ponte WSL
Traduz caminhos entre os dois mundos
```

```
# Ubuntu (WSL)
Acessa disco Windows via /mnt/c/
```

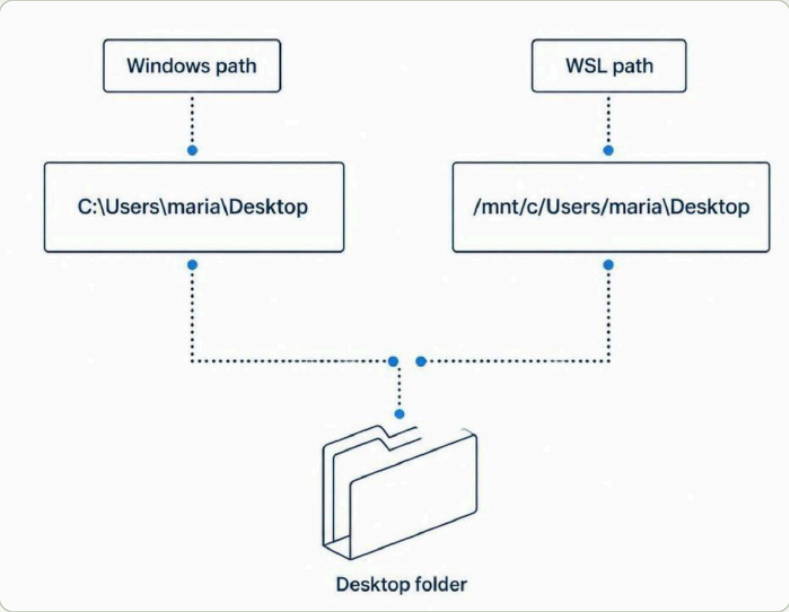
O que é um Path?

Endereço de um arquivo no sistema. Sequência de diretórios até o destino — como um endereço postal, só que com pastas.

Mesmo lugar, dois endereços. Desktop da usuária **maria**:

paths

```
# Windows
C:\Users\maria\Desktop
# WSL
/mnt/c/Users/maria/Desktop
```



Navegando com . e ..

```
# . (ponto)
Diretório atual — onde você está agora
```

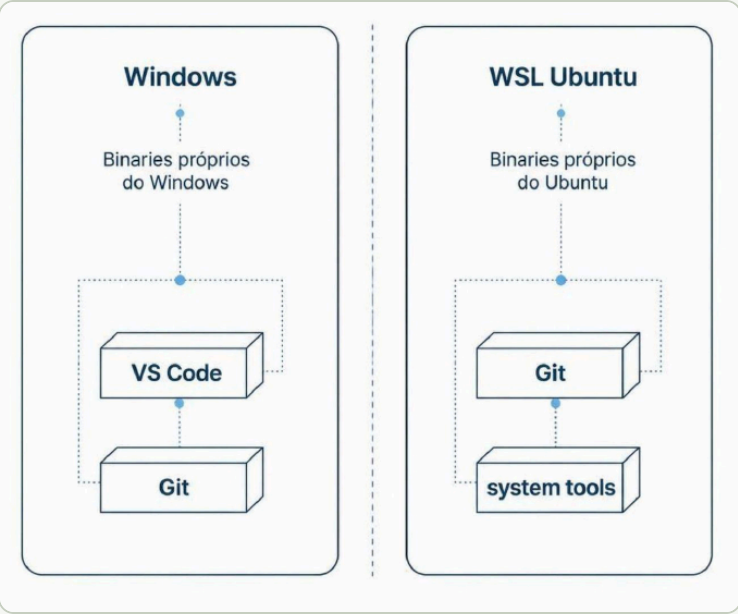
```
# .. (dois pontos)
Diretório pai — um nível acima na estrutura
```

Existem em Linux e Windows — essenciais para navegar e instalar ferramentas.

Binários e Ambientes Isolados

Cada ambiente tem seus próprios binários. Instalar o Git no Windows **não** o instala no WSL — e vice-versa.

- `mkdir`, `ls`, `cd` — já vêm instalados no Linux
- Git e VS Code precisam ser instalados manualmente
- Cada máquina exige seu próprio processo



Vim: Editor de Texto no CLI

O que é o Vim? Editor de texto no terminal CLI, **sem interface gráfica**. Nativo do Linux/Unix — não existe no Windows nativamente.

```
# Nativo do Linux
Só funciona via WSL no Windows.
```

```
# Abrindo um arquivo
vim arquivo.txt
```

! `:q!` → sair **sem salvar**
`:wq` → salvar **e** sair

Ubuntu/WSL

```
# Instalando no Ubuntu/WSL
$ sudo apt install vim -y
```

comandos essenciais

```
i      Inserir
Esc    Modo normal
:wq    Salvar e sair
:q!    Sair sem salvar
```

IDE: Por que o VS Code Importa

```
# Editor inteligente
Destaque de sintaxe, sugestões e detecção de erros
```

```
# Terminal integrado
Tudo no mesmo programa, sem alternar janelas
```

```
# Integração com Git
Controle de versão diretamente na interface
```

Instalado no Windows — é ele quem gerencia tela e janelas.

O Problema sem Controle de Versão

projeto/

```
projeto_final_v2_agora_vai
```

Cópias manuais, incompletas, sem histórico real.

O risco do "salvar como"

- → Impossível saber o que mudou entre versões
- → Sem como voltar para um estado específico
- → Qualquer mudança pode quebrar tudo

O Git resolve exatamente esse caos — de forma elegante e rastreável.

Instalando o Git

No Windows: baixar instalador em **git-scm.com** e seguir o assistente gráfico.

No WSL (Ubuntu):

Ubuntu/WSL

```
$ sudo apt update
$ sudo apt install git -y
$ git --version
```

Por que instalar nos dois? Cada ambiente é isolado. O Git do Windows não existe dentro do WSL.

Bônus: VS Code no WSL. Após instalar o VS Code no Windows, abra qualquer pasta Linux com:

pasta-linux/

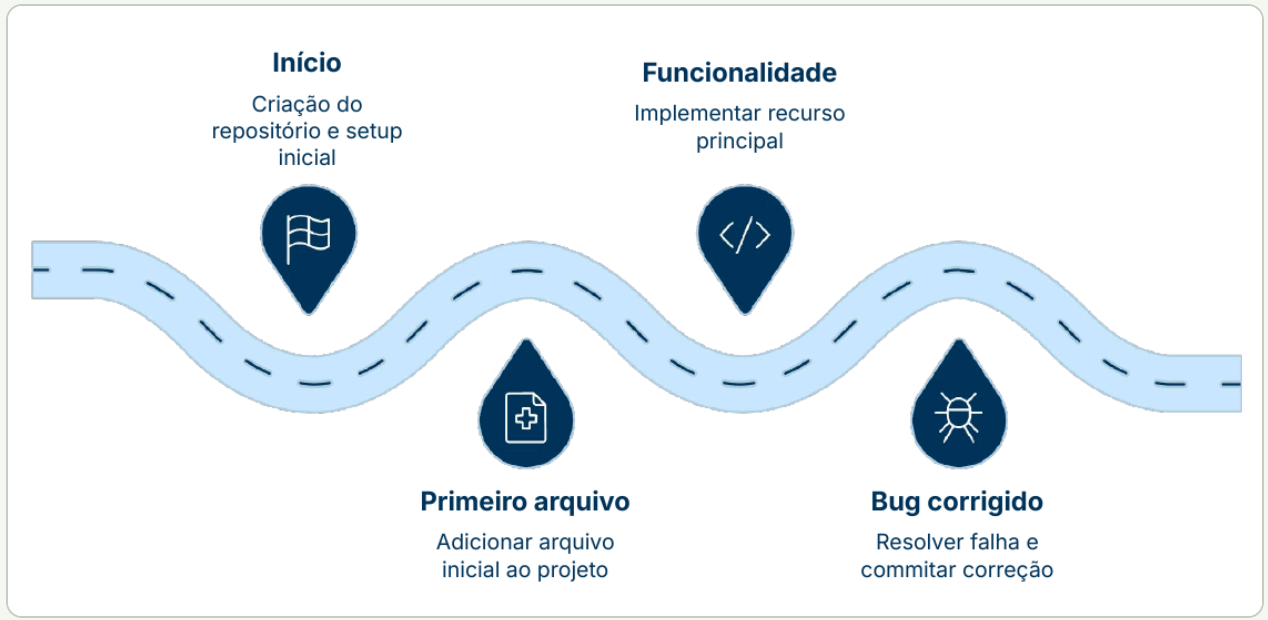
```
$ code .
```

O `.` aponta para a pasta atual.

Como o Git Funciona

Pense em fotografias. Cada **commit** é um retrato completo de todos os arquivos do projeto naquele instante.

- → Tem data, autor e mensagem explicativa
- → Pode ser revisitado a qualquer momento
- → A sequência forma a linha do tempo do projeto



Por ora: linha única, sem ramificações. *Branches* ficam para outra disciplina.

Fluxo Básico do Git



projeto/

```
$ git init
# Cria o repositório na pasta atual
$ git branch -M main
# Renomeia a branch padrão para main
$ git status
# Raio-x do momento atual (nunca altera nada)
$ git add .
# Prepara todos os arquivos modificados
$ git commit -m "mensagem clara do que mudou"
$ git log
# Exibe a linha do tempo de commits
```

Configurando o Git

Antes de fazer qualquer commit, o Git precisa saber quem você é. Esses dados aparecem em cada commit registrado no histórico do projeto.

definindo nome e e-mail

```
$ git config --global user.name "Seu Nome"
$ git config --global user.email "seu@email.com"
```

verificando as configurações salvas

```
$ git config --global user.name
$ git config --global user.email
```

`--global` aplica a configuração para todos os repositórios da sua máquina. Você só precisa fazer isso uma vez por computador.

Recuperando o Passado

`git checkout` — restaura um arquivo para como estava em um commit específico.

```
projeto/
$ git checkout a1b2c3d -- anotacoes.txt
```

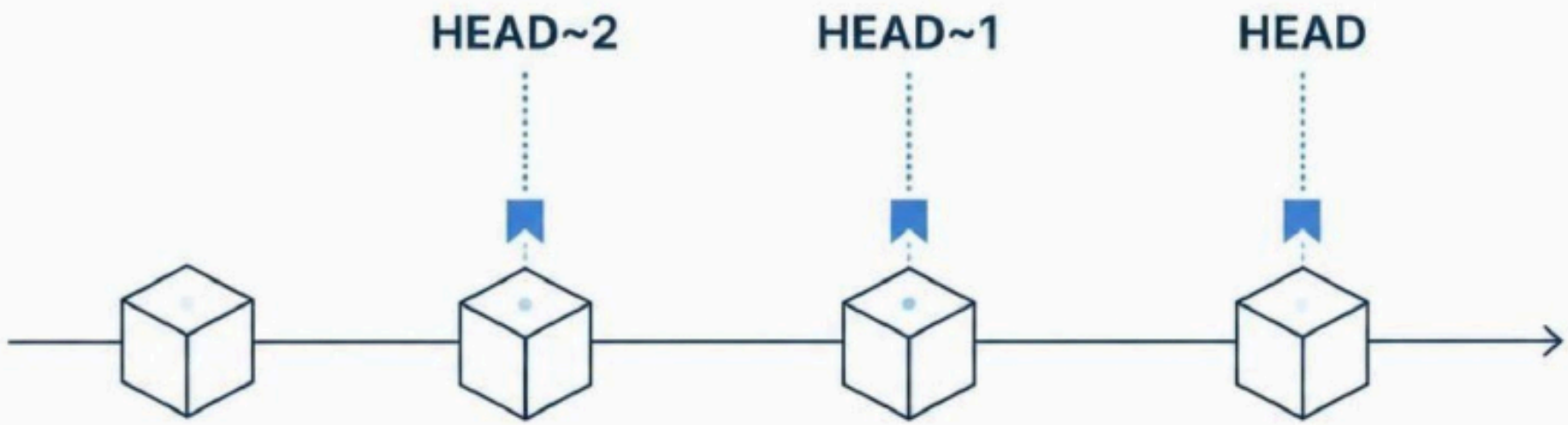
`git diff` — mostra linha a linha o que mudou desde o último commit — antes de decidir o que registrar.

```
projeto/
$ git diff
```

O histórico vive na pasta `.git`. Criada pelo `git init`, guarda todos os commits localmente.

> **i** Ainda não existe cópia em outro lugar. Isso é tarefa do GitHub — próxima aula.

HEAD: o Marcador de Página



O que é HEAD? Referência automática do Git que aponta sempre para o **commit atual** — sem precisar copiar nenhum hash.

projeto/

```
# Restaura arquivo ao estado atual
$ git checkout HEAD -- anotacoes.txt
# Um commit antes do atual
$ git checkout HEAD~1 -- anotacoes.txt
# Dois commits antes
$ git checkout HEAD~2 -- anotacoes.txt
```

Soft Delete vs. Hard Delete

Soft Delete — `--soft`
Move o HEAD para um commit anterior, mas preserva as mudanças nos arquivos.

Como guardar uma foto numa gaveta — sem rasgar nada.

```
projeto/
$ git reset --soft HEAD~1
```

Hard Delete — `--hard`
Move o HEAD para trás e descarta definitivamente todas as mudanças posteriores.

> ! Sem segunda chance. Use com cautela.

```
projeto/
$ git reset --hard HEAD~1
```

Soft e Hard Delete além do Git

Um conceito que se repete. A mesma lógica aparece em Banco de Dados:

- → **Soft delete:** marcar registro com flag `deletado=true` — dado continua na tabela, recuperável
- → **Hard delete:** comando `DELETE` — linha removida definitivamente

---

Reconhecer padrões que se repetem entre disciplinas é pensar como engenheiro de software.

Resumo da aula

- → Filesystem compartilhado WSL ↔ Windows
- → Paths e a ponte `/mnt/c/`
- → Instalação de VS Code e Git
- → Commits como linha do tempo
- → HEAD, soft e hard delete

Próxima aula: GitHub — compartilhar o histórico com o mundo.

tarefa

TF 04

docs.google.com/forms/d/e/1FAIpQLScq8Jid2BV3f8Ngbmh0XDYsnez7BO_mlcSeTf7Ku5XuB2LDxg/viewform